Chapter 1

Numerical Root-Finding Methods

1.1 Introduction

Roots of a function means the solutions of $f(x) = 0$. Except for few specific functions, it is generally not feasible to obtain an exact analytical expression for the root, thus preventing the precise determination of the solution. We know, for example, that the solution of an equation of the form

$ax^2 + bx + c = 0$ is $x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$. (1.1)

A analytic formula like this does not exist for polynomials of all degrees. Such problem can be solved by approximating a solution using an iterative method. We can solve such problems numerically using few algorithms.

1.2 Bisection Method

The bisection method is based on the Bolzano's Theorem which states that

If a continuous function has values of opposite sign inside an interval, then it has a root in that interval.

If a continuous function changes sign over an interval, we can narrow down the approximate solution by using the midpoint of the interval and searching to the left and right of it (similar to a binary search). This can be broken down into a few steps:

1. Confirm that a root lies between the interval by checking if $f(a)f(b) < 0$. Otherwise,

choose different interval.

2. Introduce a new variable that is defined as the midpoint of the interval, $x_0 = (a + b)/2$.

(1.2)

3. Check if the root lies left or right of the midpoint.

   (a) If $f(a)f(x_0) < 0$, the root lies to the left.

   (b) If $f(b)f(x_0) < 0$, the root lies to the right.

4. Repeat the process from step 2 until the width, $b - a$, is less than a very small number, for example $\epsilon = 0.001$.

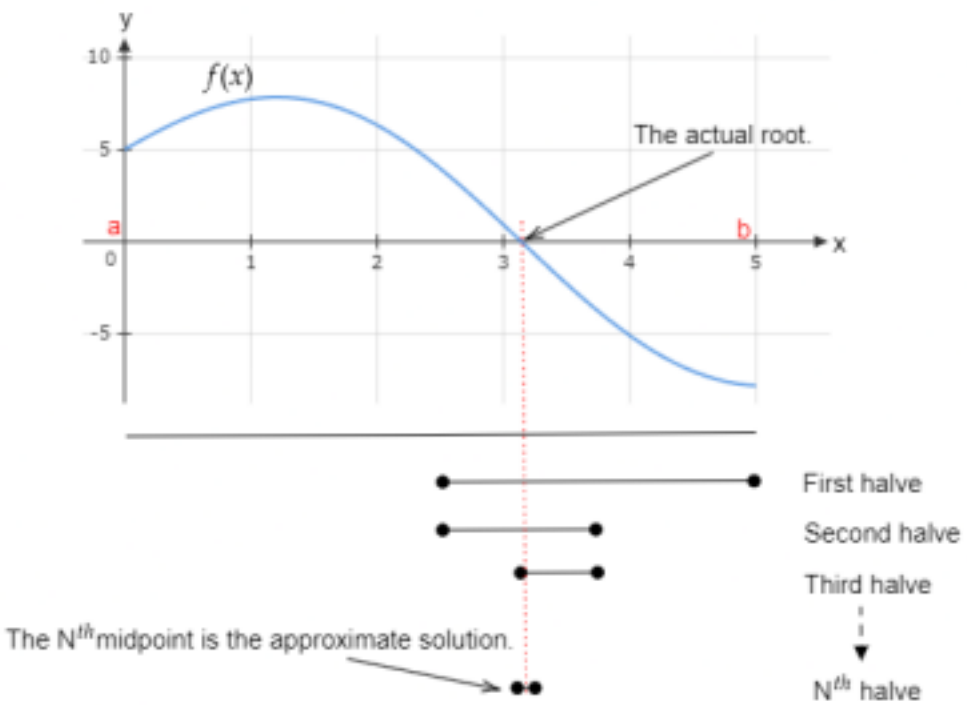

Figure 1.1: Figure: Bisection Method applied to a function $f(x)$ with initial guesses as a and b.

1.2.1 Example

Problem

Find the approximate root of the function

$$f(x) = x^3 + x^2 + x - 8 \quad (1.3)$$

within the interval [1, 2] using the bisection method with an accuracy of $\varepsilon = 0.1$.

Solution

We apply the bisection method step-by-step to narrow down the interval until it is smaller than the required accuracy.

Check if $f(x)$ has opposite signs at $x = 1$ and $x = 2$:

$$f(1) = 1^3 + 1^2 + 1 - 8 = -5$$
$$f(2) = 2^3 + 2^2 + 2 - 8 = 6$$

Since $f(1) = -5$ and $f(2) = 6$ have opposite signs, there is at least one root in ([1, 2].

We will iteratively bisect the interval and evaluate $f(x)$ at the midpoint until the interval width is smaller than $\varepsilon = 0.1$.

Iteration 1

$$x_0 = \frac{1 + 2}{2} = 1.5$$
$$f(1.5) = 1.5^3 + 1.5^2 + 1.5 - 8 = -0.875$$

Since $f(1.5) = -0.875$ and $f(2) = 6$ have opposite signs, the root is in [1.5, 2].

Iteration 2

$$x_0 = \frac{1.5 + 2}{2} = 1.75$$

$$f(1.75) = 1.75^3 + 1.75^2 + 1.75 - 8 = 2.171875$$

Since $f(1.5) = -0.875$ and $f(1.75) = 2.171875$ have opposite signs, the root is in [1.5, 1.75].

Iteration 3

$$x_0 = \frac{1.5 + 1.75}{2} = 1.625$$

$$f(1.625) = 1.625^3 + 1.625^2 + 1.625 - 8 = -0.435546875$$

Since $f(1.625) = -0.435546875$ and $f(1.75) = 2.171875$ have opposite signs, the root is in [1.625, 1.75].

Iteration 4

$$x_0 = \frac{1.625 + 1.75}{2} = 1.6875$$

$$f(1.6875) = 1.6875^3 + 1.6875^2 + 1.6875 - 8 = 0.340576171875$$

Since $f(1.625) = -0.435546875$ and $f(1.6875) = 0.340576171875$ have opposite signs, the root is in [1.625, 1.6875].

Iteration 5

$$x_0 = \frac{1.625 + 1.6875}{2} = 1.65625$$

$$f(1.65625) = 1.65625^3 + 1.65625^2 + 1.65625 - 8 = -0.0494384765625$$

$$f(1.6875) = 0.340576171875$$

Since $f(1.65625) = -0.0494384765625$ and $f(1.6875) = 0.340576171875$ have opposite signs, the root is in [1.65625, 1.6875].

Approximate Root

The interval [1.65625, 1.6875] has a width of

$$1.6875 - 1.65625 = 0.03125,$$

which is less than the required accuracy of 0.1. Therefore, we stop here and take the midpoint as an approximate root:

$$\text{Approximate root} \approx 1.671875.$$

1.2.2 Python Implementation

The implementation of this algorithm in python is the following,

```
eps = 1e-04 #0.0001

def bisection(expr,a,b):
    #Check for wrong input:
    if expr.subs(x,a)*expr.subs(x,b) >= 0:
        print("Invalid values.")
    else:
        while (b-a) >= eps:
```

```
    #Find the midpoint of a and b:
    x0 = (a+b)/2
    #set condition to see if root lies left or right
    if expr.subs(x,a)*expr.subs(x,x0) < 0:
        b = x0
    elif expr.subs(x,b)*expr.subs(x,x0) < 0:
        a = x0
  print("The root is ", x0)
  return
```


1.2.3 Exercises

(a) **Basic Check for Root (Easy)**

Given $f(x) = x^2 - 4$, use the Bisection Method to determine if there's a root between $a = 1$ and $b = 3$. Perform two iterations to find an approximate value for the root.

(b) **Intermediate Iteration (Easy-Medium)**

Apply the Bisection Method to the function $f(x) = x^3 - 6x + 2$ within the interval [1, 2]. Find the root to within $\epsilon = 0.1$, completing at least three iterations.

(c) **Precision Increase (Medium)**

Using the Bisection Method, approximate the root of $f(x) = x^2 - 5$ in the interval [2, 3]. Continue until the interval width is less than $\epsilon = 0.01$.

(d) **Nonlinear Root Approximation (Medium-Hard)**

For $f(x) = \cos(x) - x$, find an approximate root within the interval [0, 1] using the Bisection Method. Complete enough iterations to reach a precision of $\epsilon = 0.001$.

(e) **High Precision and Multiple Iterations (Hard)**

Apply the Bisection Method to approximate the root of $f(x) = x^3 - x - 1$ within the interval [1, 2] until the interval width is less than $\epsilon = 0.0001$. Determine the number of iterations needed to achieve this precision, showing each step.

1.3 Newton-Raphson Method

One can find the root of a continuous function using another method known as the Newton Raphson method. This method does not require that the root must lie within an interval, [a, b], rather an arbitrary point, x_a, is chosen and a root (if there is one) that lies close to that point will be discovered. The method starts with the point x_a and a slope at point is taken. The point where the slope crosses the $x - axis$ is found. This point becomes the new value for x_a.

Let x_a be a good estimate of r and let $r = x_a + h$. Since the true root is r, and $h = r - x_a$, the number h measures how far the estimate x_a is from the truth. Since h is 'small',

$$f(r) = f(x_a + h) \approx f(x_a) + h f'(x_a) + \frac{h^2}{2!} f''(x_a) + \frac{h^6}{6} + \ldots = 0. \quad (1.4)$$

and therefore, as x_a is an approximate and it is not equal to the root, $f(x_a)$ is close to 0,

$$h \approx -\frac{f(x_a)}{f'(x_a)}. \quad (1.5)$$

As, $r = x_a + h$, we can write

$$r = x_a - \frac{f(x_a)}{f'(x_a)}. \quad (1.6)$$

We can turn this into an iterative formula,

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}. \quad (1.7)$$

Here, $f' = \frac{df}{dx}$. This iterative process will continue until $|f(x_{n+1})| < \epsilon$.

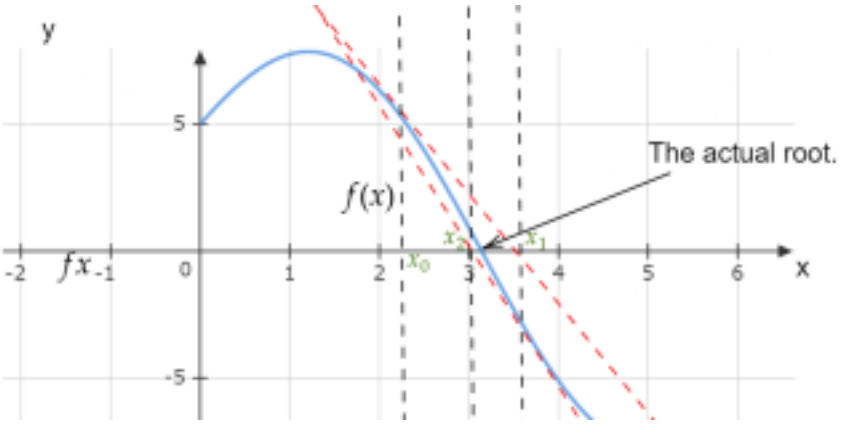

Figure 1.2: Figure: Demonstration of Newton-Raphson method. x_n moves closer to the actual root per iteration.

1.3.1 Example

Problem

Find the approximate root of the function

$$f(x) = x^2 - 4$$

starting from an initial guess of $x_0 = 5$ using Newton's method, with an accuracy of $\varepsilon = 0.1$.

7

Solution

We will apply Newton's method, which uses the formula:

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)} \tag{1.8}$$

where $f'(x)$ is the derivative of $f(x)$. For the function

$$f(x) = x^2 - 4,$$

, we have

$$f'(x) = 2x.$$

.

Starting with $x_0 = 5$, we will compute each successive approximation x_{n+1} until the difference between consecutive approximations is less than $\varepsilon = 0.1$.

Iteration 1

$$\begin{aligned} x_1 &= x_0 - \frac{f(x_0)}{f'(x_0)} \\ &= 5 - \frac{5^2 - 4}{2 \cdot 5} \\ &= 5 - \frac{21}{10} \\ &= 2.9 \end{aligned}$$

Iteration 2

$$\begin{aligned} x_2 &= x_1 - \frac{f(x_1)}{f'(x_1)} \\ &= 2.9 - \frac{2.9^2 - 4}{2 \cdot 2.9} \\ &= 2.9 - \frac{4.41}{5.8} \\ &\approx 2.1397 \end{aligned}$$

Iteration 3

$$\begin{aligned} x_3 &= x_2 - \frac{f(x_2)}{f'(x_2)} \\ &= 2.1397 - \frac{2.1397^2 - 4}{2 \cdot 2.1397} \\ &\approx 2.0062 \end{aligned}$$

Iteration 4

$$
\begin{aligned}
x_4 &= x_3 - \frac{f(x_3)}{f'(x_3)} \\
&= 2.0062 - \frac{2.0062^2 - 4}{2 \cdot 2.0062} \\
&\approx 2.00002
\end{aligned}
$$

Since the difference between $x_3 \approx 2.0062$ and $x_4 \approx 2.00002$ is less than $\varepsilon = 0.1$, we stop here.

Final Answer

The approximate root is:

$$x \approx 2.000$$

1.3.2 Python Implementation

The implementation of this is the following,

```
def NM(expr, x0):
```

```
    df = smp.diff(expr, x)
    while abs(f.subs(x,x0)) >= eps:
        r = f.subs(x,x0)/(df.subs(x,x0))
        #As r becomes really small, we eventually approach our solution. x0 -= r
    #print(x0)
    #Remove comment to see all the values explicitly
    return(x0)
```

Here, f.subs(x, obj) will replace the symbol *x* in the expression *f* with the variable *obj*. This variable can be of any type and will be replaced accordingly. If the substitute variable is an integer or float, they will undergo the arithmetic operations in the expression and give us the value. One can use sympy.lambdify() or define a lambda function and pass it as the argument instead of an expression. In that case, .subs() is not required. The derivative of a symbolic expression is taken, which is done analytically, using sympy.diff() where the first argument is the expression and the second is the variable with respect to which the derivative is taken.

1.3.3 Exercise

(a) **Simple Initial Guess (Easy)**

For $f(x) = x^2 - 2$, use Newton-Raphson Method with an initial guess $x_0 = 1.5$. Perform two iterations and calculate the approximate root.

(b) **Polynomial Root with Calculated Derivative (Easy-Medium)**

Given $f(x) = x^3 - 3x + 1$ and an initial guess $x_0 = 1$, use the Newton-Raphson Method to approximate the root. Complete at least three iterations, showing all steps.

(c) **Finding Root of Trigonometric Function (Medium)**

Apply the Newton-Raphson Method to find the root of $f(x) = \sin(x) - 0.5$ with an initial guess of $x_0 = 1$. Perform at least three iterations.

(d) **Higher Degree Polynomial (Medium-Hard)**

Use the Newton-Raphson Method to approximate a root for $f(x) = x^4 - x - 10$ with an initial guess of $x_0 = 2$. Perform four iterations and track the convergence of the solution.

(e) **Precision and Convergence Check (Hard)**

For $f(x) = e^x - 3x$ and initial guess $x_0 = 1$, use the Newton-Raphson Method to find the

root. Continue until $|f(x_n)| < 0.0001$, and determine the number of iterations needed to reach this precision.

1.4 Fixed Point Iteration Method

The Fixed Point Iteration method is an iterative process to find the solution of equations of the form:

$$x = f(x)$$

Starting with an initial guess x_0, we then generate a sequence of values:

$$x_{n+1} = f(x_n)$$

This process is repeated until the sequence converges to a fixed point, i.e., when $x_{n+1} \approx x_n$.

1.4.1 Example 1

Consider the equation $x^3 - x + 1 = 0$. We can rearrange this equation in two different ways:

- **Using** $x = g(x) = \sqrt[3]{x} - 1$**:** Starting with an initial guess x_0, the iteration becomes: $x_{n+1} =$

$$g(x_n) = \sqrt[3]{x_n} - 1$$

Now for $x_0 = -1$, the sequence becomes:

x_0	x_1	x_2	x_3	x_4	...
-1	-1.259921	-1.3122938367	-1.32235382	-1.3242687445515782	...

So this seems to converge at a point. If we iterate it for a few more steps, we will reach $x_{15} = -1.3247179572395291$, which is accurate up to 10 decimal points!

- **Using** $x = f(x) = x^3 + 1$**:** Starting with an initial guess x_0, the iteration becomes:

$$x_{n+1} = f(x_n) = x^3_n + 1$$

Now for $x_0 = -1$, the sequence becomes:

x_0	x_1	x_2	x_3	x_4	...
-1	0	1	2	9	...

So this is diverging! Why? We can guess that, since $|x^3| > 1$ for $|x| > 1$, at some point of iteration, $|x|$ exceed 1 and hence the sequence converges.

1.4.2 Example 2

Now say we want to find a real root of $f(x) = x^2 - x - 1$. From our previous experience, we may want to write it in the form $x = \sqrt{x + 1}$. And if we start from any $x_0 \geq -1$, we will approach the root $x = \frac{1+\sqrt{5}}{2}$.

However, there is another root $x = \frac{1-\sqrt{5}}{2}$, which can never be obtained using this form (why?).

So we proceed with the form $x = -\sqrt{x + 1}$ to find the negative root of $f(x)$ and the iteration is given by:

$$x_{n+1} = -\sqrt{x_n + 1}$$

Now for $x_0 = 0$, the sequence becomes:

x_0	x_1	x_2	x_3	x_4	x_5	...
0	-1	0	-1	0	-1	...

So this is neither diverging nor converging. The sequence is dancing between 0 and −1. What if we start with $x_0 = 0.5$? We will find x_1, but x_2, x_3, .. will be undefined.

However, starting with $x_0 = -0.5$ yields:

x_0	x_1	x_2	x_3	x_4	x_5
-0.5	-0.70710678	-0.5411961	-0.67735	-0.5680223	-0.65725

So the sequence seems approach $x = \frac{1-\sqrt{5}}{2} \approx -0.618034$.

Hence we conclude that $x_{n+1} = f(x_n)$ may generate a converging, diverging, or oscillating sequence (or undefined values) depending on the initial point x_0 and choice of $f(x)$. We will find out the reason soon.

1.4.3 Homework

Using the fixed point iteration method, we want to find the real solution of $x^3 + x^2 - 1 = 0$.

The actual solution is $x \approx 0.75487766624669276$.

If we express it as $x = f(x)$, we can have either $f(x) = \sqrt[3]{1 - x^2}$ or $f(x) = \sqrt{1 - x^3}$. Find x_0 for both cases to generate sequences that converge, diverge, or oscillate.

1.4.4 Visualization

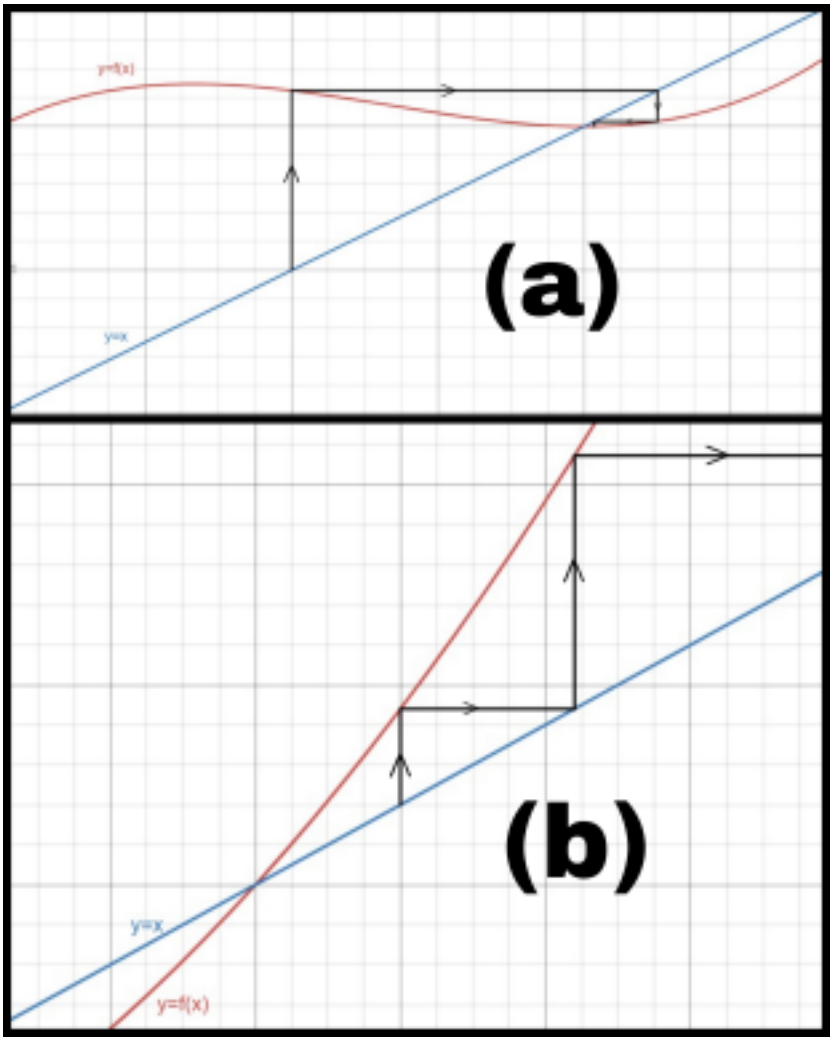

Figure 1.3: (a)Converging (b)Diverging

1.5 Banach Fixed-Point Theorem

From the previous examples, it should be apparent that f(x) plays a crucial role for the con vergence of the series. When $f(x) = 1 + x^3$, the sequence diverges, but $f(x) = \sqrt[3]{x} - 1$ make the sequence convergence. Why is so? The later seems to “contract” the sequence leading to a fixed point. We call $\phi : [a, b] \to [a, b]$ a contraction map if

$$|\phi(x) - \phi(y)| \leq k|x - y|, \ \forall x, y \in [a, b]$$

Where $0 \leq k < 1$ is called Lipschitz constant. We call $x = x_*$ a fixed point if $f(x_*) = x_*$.

From the definitions, it should be clear that the fixed point is a solution to $x = f(x)$. Now the **Banach fixed-point theorem** states that every contraction mapping on a non-empty complete metric space has a unique fixed point, and for any x_0, the sequence $x_{n+1} = f(x_n)$ converges to that fixed point, i.e.,

$$x_* = \phi \quad \lim_{n\to\infty} x = \phi(x_*)$$
$$n$$

I won't bore you with the proof but let's define $\epsilon_k = x_k - x_*$, where x_*. Now,

$$x_{k+1} = \epsilon_{k+1} + x_* = \phi(x_k) = \phi(\epsilon_k + x_*)$$

Using Taylor expansion, $\phi(\epsilon_k + x_*) = \phi(x_*) + \phi'(x_*)\epsilon_k + O(\epsilon^2{}_k)$, and since $\phi'(x_*) = x_*$, we have,

$$\epsilon_{k+1} = \phi'(x_*)\epsilon_k + O(\epsilon^2{}_k) \approx \phi'(x_*)\epsilon_k$$

Note that $\epsilon_k$is the error in each step, which will approach 0 if $|\phi'(x_*)| < 1$.
The Newton's method is given by:

$$x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)}$$

Let, $g(x) = x - \frac{f(x)}{f'(x)}$, then $x_{k+1} = g(x_k)$, which is just fixed point iteration!